

Citizen-Dev App Sandbox — Friday One-Pager

Sep 23, 2026 · @Hemant

Situation

Business users are building working prototypes in Claude Cowork and have nowhere safe to run them. A recent example is a static content site that combines several useful things into one tool; the business wants to see it running to decide whether it deserves a real development team. Today the only options are "run it on a laptop" or "put it on the network with no controls" — neither is acceptable.

The builders are citizen developers. They do not know what npm, Terraform, or a database is, and they should not have to. Cowork defaults to Python for whatever it generates, with React/JavaScript as an edge case when a user asks for it explicitly.

The decision is not "how do we host these apps." It is "how do we let untrusted prototypes be *seen* by the business without ever letting them be *trusted* by the network."

Requirement

Input: the artifacts a citizen developer generates in Cowork. **Output:** a hosted, SSO-protected application the business can open, with access to exactly what it needs and nothing more. If the business says yes, the same artifact must be promotable to a Level 2 app without a rebuild.

Out of scope for the pilot: production hosting, real data, Windows workloads, mobile targets, and anything that requires the citizen developer to touch Helm, Terraform, or a pipeline.

Recommendation

A four-stage conveyor: shape the app at generation time, enforce at build time, isolate at run time, promote only what passes. Compliance moves left to the moment Cowork writes the code; the sandbox verifies rather than repairs.

flowchart LR

```

  A[Cowork skill<br/>generates scaffold + manifest] --> B[GitHub
repo<br/>from template]
  B --> C[CI gate<br/>manifest lint · dep allowlist · SCA]
  C --> D[Sandbox namespace<br/>Helm · default-deny · SSO]
  D --> E{Business review}
  E -->|yes| F[Promote to Level 2<br/>same repo, no rebuild]
  E -->|no / 30 days| G[Namespace deleted]

```

The existing Helm-based AKS deployment path is reused for stage 4. One parametric chart, owned by platform; an agent fills `values.yaml` from the manifest and opens a PR. The agent never authors templates.

Environment: a dedicated namespace per app on the existing non-prod AKS cluster. vCluster is the phase-2 target once Nick's cost case has enough workloads behind it and CIO-level licensing is settled.

Design decisions

#	Decision	What it buys	What it rules out
1	Constrain the input, don't detect it. A MetLife Cowork skill scaffolds every app: approved stack (Python/FastAPI or static site), curated dependency list, <code>app.yaml</code> manifest, no secrets in code.	Citizen devs never learn npm; the build is predictable	Free-form "whatever the AI chose" builds
2	Deterministic build, agent only where needed. Buildpacks or pinned base images pick the image from the file tree. The agent extracts the manifest and drafts the policy allowlist for human review.	Reproducible images; the agent's output is reviewable	An agent choosing base images by inspection

#	Decision	What it buys	What it rules out
3	One chart, agent fills values. Platform owns a single parametric Helm chart. The agent produces <code>values.yaml</code> from the manifest and opens a PR.	A ten-line diff a reviewer can read	Agent-generated templates
4	Namespace-per-app, default-deny at two layers. Kubernetes NetworkPolicy (CNI) blocks all egress except DNS and all ingress except the gateway; Istio AuthorizationPolicy enforces L7 on top. Non-root, read-only rootfs, 250m/512Mi quota to start.	Isolation that holds even for traffic that bypasses the sidecar	Istio-only isolation
5	30-day TTL, born promotable. Namespace auto-deletes unless a business owner claims it. A yes means the same repo moves through the Level 2 path.	No orphaned compute; no rebuild on success	Throwaway prototypes that get rewritten

Network policy engine: use whatever the target non-prod cluster already runs. For a new cluster, Cilium (Microsoft's recommended path; Azure NPM retires on Linux September 30, 2028, Windows September 30, 2026). Calico only if Windows nodes are present.

Source: [AKS network policies](#).

Pilot plan

Two apps, two weeks, and a negative-test suite that proves the deny-all holds before any real app goes in.

Phase	Days	Work	Gate (do not cross until it passes)
0 · Prereqs	1-2	Confirm target cluster's network policy engine and K8s version (1.30+). Confirm what Cowork emits for a default "build me an app" prompt. Template repo with branch protection and CODEOWNERS.	Cluster facts documented; one real Cowork output in hand
1 · Foundation	3-5	Parametric Helm chart, <code>app.yaml</code> schema, CI gate (manifest lint, allowlist, SCA), Istio + NetworkPolicy defaults.	<code>helm lint</code> clean; gate rejects a bad manifest and an unlisted dependency
2 · Negative tests	6-7	Deploy a probe pod into a sandbox namespace and prove it cannot reach another namespace, the internet, or any internal API; prove ingress fails without SSO. Committed as a rerunnable script.	Every negative test fails the way it should
3 · App 1	8-9	The static content site from Daniel's example. nginx image, zero egress, no DB.	Business reviewer opens it via SSO on a corporate device
4 · App 2	10-12	A default Python web app from Cowork. Python base image + uvicorn, DNS-only egress, ephemeral Postgres in-namespace with a no-real-data rule.	Same as above, plus Hubble/flow logs show the app's actual egress attempts
5 · Drill	13-14	Deliberate edge case: an app that tries to call an internal API. Show it is denied and that the denial is visible to the reviewer. TTL expiry rehearsed once.	Denial observed, not assumed; namespace deleted on expiry

Non-negotiable gates: phase 2 (negative tests) and phase 5 (the drill). Skipping either means the isolation claim is an opinion.

Alternatives considered

Option	Real strengths	Why not first
vCluster now	Control-plane isolation (own CRDs, RBAC); aligns with Nick's cost-savings direction; Subu and Kalyan already have it running	Citizen apps need a namespace, not a control plane. Licensing is per CIO and onboarding is friction the pilot doesn't need. Phase 2, not phase 1.
Azure Container Apps / Static Web Apps	Fastest path to a URL; no cluster to operate; cheap for static sites	Outside the existing Istio/policy/Helm model and audit story; two platforms to govern instead of one
PR preview environments in the existing pipeline	Proven pattern; reuses everything	Citizen devs have no repos in the pipeline yet; the Cowork skill + template repo creates that on-ramp, after which this becomes the same thing
Istio-only isolation	One tool, L7 policy, already deployed	Only governs traffic through the sidecar. Needs a CNI-level NetworkPolicy as the second wall.

Things that will bite

Failure mode	Symptom	Defense
Skill and gate disagree on the allowlist	App generates cleanly, then fails CI with an error the citizen dev can't interpret; adoption dies quietly	One <code>allowlist.yaml</code> in Git, read by both the skill and the gate

Failure mode	Symptom	Defense
Build pulls from the open internet	SCA passes on what it sees; unvetted transitive packages arrive anyway	Builds resolve only through the Artifactory proxy; direct egress from build runners blocked
Cowork emits an unexpected stack	React app, mobile target, or a mix of scripts with no entrypoint	Gate rejects with a plain-language message and a link back to the skill; pilot supports static + Python only
App needs data to be reviewable	Reviewer sees an empty screen and says no for the wrong reason	Ephemeral Postgres with seed data the skill generates; no real data, ever
Istio sidecar not injected	Pod runs, AuthorizationPolicy silently does nothing	Namespace label enforced by the chart; negative test checks for the sidecar
TTL deletes an app mid-review	Reviewer's link 404s	Claim button extends TTL; expiry warning sent 5 days out

Resourcing and open questions

Minimum team for the two-week pilot: one platform engineer (chart, network policy, CI gate, negative tests) plus Hemant (Cowork skill, manifest schema, promotion criteria). Anything smaller means the negative-test suite gets skipped.

Open questions:

- ☐ Who leads: Prashant or Hemant? The two calls named different owners.
- ☐ What does "Level 2" require — criticality, data classification, support model, or all three? This defines the promotion gate.
- ☐ Which network policy engine and Kubernetes version is the target non-prod cluster on today?
- ☐ Can builds be forced through the Artifactory proxy for Python and npm, and will security allow proxying of unvetted packages?
- ☐ Is there an existing internal gateway with Entra SSO the sandbox can sit behind, or does the pilot stand one up?

Decisions requested Friday

1. Approve the pilot scope: static site + Python web app only, namespace isolation, two weeks.
2. Confirm the workstream owner and assign one platform engineer.
3. Agree that vCluster is phase 2, contingent on the cost case, not a pilot dependency.
4. Agree that the Cowork skill is in scope — constraining the input is half the solution.
5. Name who defines the Level 2 promotion criteria and by when.